

Data reuploading using PQC layer(s)

<https://github.com/tensorflow/quantum/issues/468>

ROW 62

Data reuploading using PQC layer(s) #468

New issue



Closed



luuk-visser opened on Feb 2, 2021

...

I'm trying extend my parametrized quantum circuit to use data re-uploading, as proposed by [Pérez-Salinas et al.](#)

Currently, my model exists of a single Input layer and a single PQC layer. I would like to split my single PQC layer into multiple layers such that the data uploading layers can be inserted in between the PQC layers. As `tfq.layers.PQC` requires a measurement, I'm required to change the PQC layers to something else.

However, the alternatives (State and maybe `AddCircuit`) do not accept a differentiator kwarg, and as I'm experimenting with different types of layers I would like to refrain from implementing a custom gradient for each of these. What are currently my best options to implement this?

Thanks in advance!



Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



ghellistern on Feb 2, 2021

...

Hi Luuk,

I struggled with the same Problem; have a look at issue

[#267](#) PQC in the middle of network, contd.

There a solution is presented which helps to implement the data reuploading approach... If you are interested to discuss further just tell me.

All the best
Gerhard



luuk-visser on Feb 3, 2021 · edited by luuk-visser

Edits

Author

...

Edit: updated code and given error

Hi Gerhard, thank you for your suggestion!

The problem I'm currently having is that I'm trying to insert Input layers (to input encoding circuits) in between parametrized quantum layers. Based on the solution to [#267](#), I created a small example of what I'm trying to achieve:

```
import cirq
import sympy
import numpy as np
import tensorflow as tf
import tensorflow_quantum as tfq

def param_layer(qubit, curr_depth):
    theta = sympy.Symbol(str(curr_depth))
    return cirq.Circuit(cirq.rz(theta).on(qubit))

class customPQC(tf.keras.layers.Layer):
    def __init__(self, qubit, n_layers=2):
        super().__init__(customPQC)
        self.n_layers = n_layers
        self.qubit = qubit
        self.symbols = [sympy.Symbol(str(i)) for i in range(n_layers)]

    def build(self, input_shape):
        self.managed_weights = self.add_weight(
            shape=(1, len(self.symbols)),
            initializer=tf.keras.initializers.RandomUniform(0, 2 * np.pi))

    def call(self, encoding_tfqc):
        # Create circuit_tensor alternating between data-encoding layers and parametrized layers
        circuit_tensor = encoding_tfqc
```



```

        initializer=tf.keras.initializers.RandomUniform(0, 2 * np.pi))

    def call(self, encoding_tfcirc):
        # Create circuit_tensor alternating between data-encoding layers and parametrized layers
        circuit_tensor = encoding_tfcirc

        for curr_depth in range(self.n_layers):
            if curr_depth > 0:
                circuit_tensor = tfq.layers.AddCircuit()(
                    circuit_tensor,
                    append=encoding_tfcirc
                )
            circuit_tensor = tfq.layers.AddCircuit()(
                circuit_tensor,
                append=tfq.convert_to_tensor([param_layer(self.qubit, curr_depth)])
            )

        ops = cirq.Z(self.qubit)
        return tfq.layers.Expectation()(
            circuit_tensor,
            operators=ops,
            symbol_names=self.symbols,
            symbol_values=self.managed_weights
        )

qubit = cirq.GridQubit(0,0)

X = 2*np.random.rand(10) - 1
y = X**2
X_circuit = [cirq.Circuit(cirq.ry(np.arcsin(x)).on(qubit)) for x in X]
X_tfcirc = tfq.convert_to_tensor(X_circuit)

input_layer = tf.keras.Input(shape=(), dtype=tf.string)
output_layer = customPQC(qubit)(input_layer)
model = tf.keras.Model(inputs=input_layer, outputs=output_layer)
model.compile(optimizer='Adam', loss='mse')
model.fit(X_tfcirc, y)

```

However, this gives me the following error:

```

-----
InvalidArgumentError                                Traceback (most recent call last)
<ipython-input-8-00d4b3416e42> in <module>
    56 model = tf.keras.Model(inputs=input_layer, outputs=output_layer)
    57 model.compile(optimizer='Adam', loss='mse')
--> 58 model.fit(X_tfcirc, y)

~/venv/lib64/python3.6/site-packages/tensorflow/python/keras/engine/training.py in _method_wrapper(self, *args, **kwargs):
    106 def _method_wrapper(self, *args, **kwargs):
    107     if not self.in_multi_worker_mode(): # pylint: disable=protected-access
--> 108     return method(self, *args, **kwargs)
    109
    110     # Running inside 'run_distribute_coordinator' already.

~/venv/lib64/python3.6/site-packages/tensorflow/python/keras/engine/training.py in fit(self, x, y, batch_size,
    1096 batch_size=batch_size):
    1097     callbacks.on_train_batch_begin(step)
--> 1098     tmp_logs = train_function(iterator)
    1099     if data_handler.should_sync:
    1100         context.async_wait()

~/venv/lib64/python3.6/site-packages/tensorflow/python/eager/def_function.py in __call__(self, *args, **kwargs)
    778 else:
    779     compiler = "nonXla"
--> 780     result = self._call(*args, **kwargs)
    781
    782     new_tracing_count = self.get_tracing_count()

~/venv/lib64/python3.6/site-packages/tensorflow/python/eager/def_function.py in _call(self, *args, **kwargs)
    838     # Lifting succeeded, so variables are initialized and we can run the
    839     # stateless function.
--> 840     return self._stateless_fn(*args, **kwargs)
    841 else:
    842     canon_args, canon_kwds = \

~/venv/lib64/python3.6/site-packages/tensorflow/python/eager/function.py in __call__(self, *args, **kwargs)
    2827     with self._lock:

```

```

842     canon_args, canon_kws = \

~/venv/lib64/python3.6/site-packages/tensorflow/python/eager/function.py in __call__(self, *args, **kwargs)
2827     with self._lock:
2828         graph_function, args, kwargs = self._maybe_define_function(args, kwargs)
-> 2829     return graph_function._filtered_call(args, kwargs) # pylint: disable=protected-access
2830
2831     @property

~/venv/lib64/python3.6/site-packages/tensorflow/python/eager/function.py in _filtered_call(self, args, kwargs,
1846         resource_variable_ops.BaseResourceVariable)]]],
1847         captured_inputs=self.captured_inputs,
-> 1848         cancellation_manager=cancellation_manager)
1849
1850     def _call_flat(self, args, captured_inputs, cancellation_manager=None):

~/venv/lib64/python3.6/site-packages/tensorflow/python/eager/function.py in _call_flat(self, args, captured_in
1922     # No tape is watching; skip to running the function.
1923     return self._build_call_outputs(self._inference_function.call(
-> 1924         ctx, args, cancellation_manager=cancellation_manager))
1925     forward_backward = self._select_forward_and_backward_functions(
1926         args,

~/venv/lib64/python3.6/site-packages/tensorflow/python/eager/function.py in call(self, ctx, args, cancellation
548         inputs=args,
549         attrs=attrs,
--> 550         ctx=ctx)
551     else:
552         outputs = execute.execute_with_cancellation(

~/venv/lib64/python3.6/site-packages/tensorflow/python/eager/execute.py in quick_execute(op_name, num_outputs,
58     ctx.ensure_initialized()
59     tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
--> 60         inputs, attrs, num_outputs)
61 except core._NotOkStatusException as e:
62     if name is not None:

InvalidArgumentError: programs and programs_to_append must have matching sizes.
[[node functional_11/custom_pqc_6/add_circuit/TfqAppendCircuit (defined at <string>:65) ]] [Op:__infe

Errors may have originated from an input operation.
Input Source operations connected to node functional_11/custom_pqc_6/add_circuit/TfqAppendCircuit:
functional_11/custom_pqc_6/Const (defined at /home/s1707264/venv/lib64/python3.6/site-packages/tensorflow_que
functional_11/Squeeze (defined at <ipython-input-8-00d4b3416e42>:58)

Function call stack:
train_function

```

Even if this solution were to work, it rebuilds the entire circuit for every single call made, which seems to me like unnecessary overhead. Any suggestions to fix and/or improve upon this?



ghellstern on Feb 4, 2021 · edited by ghellstern

Edits ...

Hi Luuk,
I'm a little bit confused about your code - putting everything together in one class is maybe possible but at least for me too complicated :-)
See below the code which implements data-reuploading in a 1-dim toy example. Extending it to more qubits and higher dimensional data is straightforward. The only trick needed is the splitting layer. I worked out an example which uses data reuploading for MNIST-data and up to 15 Qubits and it really works ;-)
By adding classical layers before and after the quantum network you can further condense and transform the input data and the measurement results which is quite useful imho....
Good luck ! Gerhard

```

` import tensorflow as tf
import tensorflow_quantum as tfq
import cirq
import sympy
import numpy as np
import matplotlib.pyplot as plt

class SplitBackpropQ(tf.keras.layers.Layer):

```

```
def __init__(self, upstream_symbols, managed_symbols, managed_init_vals,
             operators):
    """Create a layer that splits backprop between several variables.
```

Args:

```
    upstream_symbols: Python iterable of symbols to backprop
                      through this layer.
    managed_symbols: Python iterable of symbols to backprop
                     into variables managed by this layer.
    managed_init_vals: Python iterable of initial values
                       for managed_symbols.
    operators: Python iterable of operators to use for expectation.
```

```
    """
```

```
    super().__init__(SplitBackpropQ)
    self.all_symbols = upstream_symbols + managed_symbols
    self.upstream_symbols = upstream_symbols
    self.managed_symbols = managed_symbols
    self.managed_init = managed_init_vals
    self.ops = operators
```

```
def build(self, input_shape):
    self.managed_weights = self.add_weight(
        shape=(1, len(self.managed_symbols)),
        initializer=tf.constant_initializer(self.managed_init))
```

```
def call(self, inputs):
    ## inputs are: circuit tensor, upstream values
    upstream_shape = tf.gather(tf.shape(inputs[0]), 0)
    tiled_up_weights = tf.tile(self.managed_weights, [upstream_shape, 1])
    joined_params = tf.concat([inputs[1], tiled_up_weights], 1)
    return tfq.layers.Expectation()(inputs[0],
                                     operators=self.ops,
                                     symbol_names=self.all_symbols,
                                     symbol_values=joined_params)
```

```
##Create One-Dimensional-Data for Classification
```

```
np.random.seed(seed=123)
```

```
n = 900
```

```
data = np.random.rand(n, 1)
```

```
labels = []
```

```
for p in range(0, n):
```

```
    if data[p] <= 0.5:
```

```
        label = [1, 0]
```

```
    else:
```

```
        label = [0, 1]
```

```
    labels.append(label)
```

```
labels = np.array(labels, dtype=np.int32)
```

```
bit = cirq.GridQubit(0, 0)
```

```
symbols = sympy.symbols('alpha, beta, gamma, eta')
```

```
ops = [cirq.Z(bit)]
```

```
circuit = cirq.Circuit( ##Data is encoded via first Y-rotation:
```

```
    cirq.Y(bit)**symbols[0],
```

```
    cirq.Y(bit)**symbols[1],
```

```
    cirq.Z(bit)**symbols[2],
```

```
    cirq.Y(bit)**symbols[3],
```

```
    ##Adding the data-encoding again corresponds to data-reuploading:
```

```
    cirq.Y(bit)**symbols[0],
```

```
)
```

```
data_input = tf.keras.Input(shape=(1,), dtype=tf.dtypes.float32)
```

```
##Use a classical NN to transform the data
```

```
encod_1 = tf.keras.layers.Dense(10, activation=tf.keras.activations.relu)(data_input)
```

```
encod_2 = tf.keras.layers.Dense(1, activation=tf.keras.activations.sigmoid)(encod_1)
```

```
unused = tf.keras.Input(shape=(), dtype=tf.dtypes.string)
```

```
expectation = SplitBackpropQ(['alpha'], ['beta', 'gamma', 'eta'], [np.pi / 2, np.pi/2, np.pi/2 ],
ops)([unused, encod_2])
```

```
expectation = SplitBackpropQ(['alpha'], ['beta', 'gamma', 'eta'], [np.pi / 2, np.pi/2, np.pi/2 ],
ops)([unused, encod_2])
```

```
classifier = tf.keras.layers.Dense(2, activation=tf.keras.activations.softmax)
classifier_output = classifier(expectation)
```

```
model = tf.keras.Model(inputs=[unused, data_input], outputs=classifier_output)
```

```
tf.keras.utils.plot_model(model, show_shapes=True, dpi=70)
```

```
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
loss=tf.keras.losses.mean_squared_error)
##model.compile(optimizer='Adam', loss='mse')
```

```
history=model.fit([tfq.convert_to_tensor([circuit for _ in range(n)]), data],
labels, batch_size=10,
epochs=100)
print(model.trainable_variables)
```

```
plt.plot(history.epoch, history.history['loss'])
```



luuk-visser on Feb 4, 2021

Author ...

Hi Gerhard, after trying to get this to work for a long time I just realized I might as well use instances of the entire circuit (including parametrized layers) as input for the model. My problem was the fact that I wanted to represent only the data-encoding part of the circuit as input to the model, which I guess would be more elegant but I'm already happy to finally have my code running now. Thanks for your help!



luuk-visser closed this as **completed** on Feb 4, 2021



MattWard97 on Oct 27, 2022

...

Hi Gerhard, after trying to get this to work for a long time I just realized I might as well use instances of the entire circuit (including parametrized layers) as input for the model. My problem was the fact that I wanted to represent only the data-encoding part of the circuit as input to the model, which I guess would be more elegant but I'm already happy to finally have my code running now. Thanks for your help!

@luuk-visser Doesn't The tensorflow-quantum PQC layer documentation say that you can not have free parameters in the input to the PQC layer? "In order to extract information from our circuit, we must apply measurement operators. For now we choose to make a Z measurement. In order to observe an output, we must also feed our model quantum data (NOTE: quantum data means quantum circuits with no free parameters)." So you are saying your input to the PQC layer DOES have free sympy parameters / weights? Did this work for you? Thanks



lockwo on Oct 27, 2022

Contributor ...

My goto method for data reuploading is to use some free parameters (for reuploading) and some non-free by using the ControlledPQC layer. You can see examples of how to do this here:

https://www.tensorflow.org/quantum/tutorials/quantum_reinforcement_learning,
https://github.com/lockwo/quantum_computation/blob/master/TFQ/data_reupload/uat.py,
https://github.com/lockwo/quantum_computation/blob/master/TFQ/data_reupload/reup.py.

Regarding your specific question, yes the input to the PQC should be unbound parameter free.



jaeyoo added a commit that references this issue on Mar 30, 2023

Merge pull request [tensorflow#468](#) from quantuilib/unpin-noisy-qsimcir

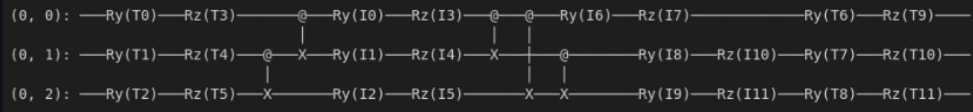
Verified aa5b175



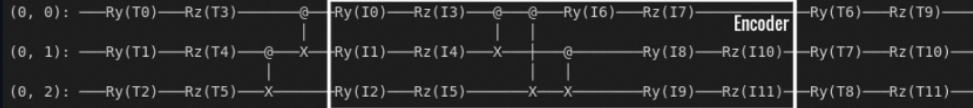
anikde on Sep 5, 2023 · edited by anikde

Edits ...

I am facing a similar problem, my parameterized circuit is a 3-qubit model



in which I am using an encoder circuit shown the figure below.



. The trainable parameters are suffixed with T and input parameters are prefixed with I.

The encoder circuit can encode data dimension of (12,).

So, I know the input to the model input shape should be of size (12,). But since my circuit is of 3-qubit mode, I need to make changes in to the call function to make it work, particularly in `tf.einsum` (probably).

The code below is a custom_layer class from [tensorflow_quantum_reinforcement_learning_notebook](#), however I changed the model circuit as per my requirement.

```
class ReUploadingPQC(tf.keras.layers.Layer):
    """
    Performs the transformation (s_1, ..., s_d) -> (theta_1, ..., theta_N, lmbd[1][1]s_1, ..., lmbd[1][M]s_1,
    ....., lmbd[d][1]s_d, ..., lmbd[d][M]s_d) for d=input_dim, N=theta_dim and M=n_layers.
    An activation function from tf.keras.activations, specified by 'activation' ('linear' by default) is
    then applied to all lmbd[i][j]s_i.
    All angles are finally permuted to follow the alphabetical order of their symbol names, as processed
    by the ControlledPQC.
    """
    def __init__(self, qubits, n_layers, observables, activation="linear", name="re-uploading_PQC"):
        super(ReUploadingPQC, self).__init__(name=name)
        self.n_layers = n_layers
        self.n_qubits = len(qubits)
        self.observables = observables

        circuit, theta_symbols, input_symbols = generate_circuit(qubits, n_layers)
        self.circuit = circuit

        theta_init = tf.random_uniform_initializer(minval=0.0, maxval=np.pi)
        self.theta = tf.Variable(
            initial_value=theta_init(shape=(1, len(theta_symbols)), dtype="float32"),
            trainable=True, name="thetas"
        )

        lmbd_init = tf.ones(shape=(self.n_qubits * self.n_layers,))
        self.lmbd = tf.Variable(
            initial_value=lmbd_init, dtype="float32", trainable=True, name="lmbdas"
        )

        # Define explicit symbol order.
        symbols = [str(symb) for symb in theta_symbols + input_symbols]
        self.indices = tf.constant([symbols.index(a) for a in sorted(symbols)])

        self.activation = activation
        self.empty_circuit = tfq.convert_to_tensor([cirq.Circuit()])
        self.computation_layer = tfq.layers.ControlledPQC(self.circuit, self.observables)

    def call(self, inputs):
        # inputs[0] = encoding data for the state.
        batch_dim = tf.gather(tf.shape(inputs[0]), 0)
        tiled_up_circuits = tf.repeat(self.empty_circuit, repeats=batch_dim)
        tiled_up_thetas = tf.tile(self.theta, multiples=[batch_dim, 1])
        tiled_up_inputs = tf.tile(inputs[0], multiples=[1, self.n_layers])
        scaled_inputs = tf.einsum("i,ji->ji", self.lmbd, tiled_up_inputs)
        squashed_inputs = tf.keras.layers.Activation(self.activation)(scaled_inputs)
```



```

        tiled_up_thetas = tf.tile(self.thetas, multiples=[batch_size, 1])
        tiled_up_inputs = tf.tile(inputs[0], multiples=[1, self.n_layers])
        scaled_inputs = tf.einsum("i,ji->ji", self.lmbd, tiled_up_inputs)
        squashed_inputs = tf.keras.layers.Activation(self.activation)(scaled_inputs)

        joined_vars = tf.concat([tiled_up_thetas, squashed_inputs], axis=1)
        joined_vars = tf.gather(joined_vars, self.indices, axis=1)

        return self.computation_layer([tiled_up_circuits, joined_vars])

```

Then when I try to create model with following lines of code

```

def generate_model_policy(qubits, n_layers, n_actions, beta, observables):
    """Generates a Keras model for a data re-uploading PQC policy."""

    input_tensor = tf.keras.Input(shape=(12, ), dtype=tf.dtypes.float32, name='input')
    re_uploading_pqc = ReUploadingPQC(qubits, n_layers, observables)(input_tensor)
    process = tf.keras.Sequential([
        Alternating(n_actions),
        tf.keras.layers.Lambda(lambda x: x * beta),
        tf.keras.layers.Dense(3, activation="softmax"),
    ], name="observables-policy")
    policy = process(re_uploading_pqc)
    model = tf.keras.Model(inputs=[input_tensor], outputs=policy)

    return model

model = generate_model_policy(qubits, n_layers, n_actions, 1.0, observables)
The error is

```

Exception encountered when calling layer "re-uploading_PQC" (type ReUploadingPQC).

in user code:

```

File "/tmp/ipykernel_1109761/1735579080.py", line 45, in call *
    scaled_inputs = tf.einsum("i,ji->ji", self.lmbd, tiled_up_inputs)

ValueError: Dimensions must be equal, but are 3 and 12 for '{{node re-uploading_PQC/einsum/Einsum}} = Einsum[

```

Call arguments received:

• inputs=['tf.Tensor(shape=(None, 12), dtype=float32)']

I didn't understand how tf.einsum works here. Can someone help me how to make it work.



lockwo on Sep 6, 2023

Contributor ...

Einsum should be fine here, as long as the dimensions are correct (if you want to understand einsum read: <https://rockt.github.io/2018/04/30/einsum>). It appears as though the issue is with shapes. Lambda assumes structure that isn't true. If you look in the tutorial, there are 1 param per 1 qubit for the lambda parameters. However, you want 3 lambda parameters per qubit because the ansatz is different. So the lambda is creating the wrong number of parameters (which then errors when you feed it into the model creation)



1



anikde on Sep 7, 2023

...

Thanks, I didn't count the number of parameters in the `lmbd_init` variable.

```

n_qubits = 3
n_layers = 1

```


in user code:

```
File "/tmp/ipykernel_1109761/1735579080.py", line 45, in call *
scaled_inputs = tf.einsum("i,ji->ji", self.lmbd, tiled_up_inputs)

ValueError: Dimensions must be equal, but are 3 and 12 for '{{node re-uploading_PQC/einsum/Einsum}} = Einsum[1
```

Call arguments received:

- inputs=['tf.Tensor(shape=(None, 12), dtype=float32)']

I didn't understand how tf.einsum works here. Can someone help me how to make it work.



lockwo on Sep 6, 2023

Contributor ...

Einsum should be fine here, as long as the dimensions are correct (if you want to understand einsum read: <https://rockt.github.io/2018/04/30/einsum>). It appears as though the issue is with shapes. Lambda assumes structure that isn't true. If you look in the tutorial, there are 1 param per 1 qubit for the lambda parameters. However, you want 3 lambda parameters per qubit because the ansatz is different. So the lambda is creating the wrong number of parameters (which then errors when you feed it into the model creation)

👍 1



anikde on Sep 7, 2023

...

Thanks, I didn't count the number of parameters in the `lmbd_init` variable.

```
n_qubits = 3
n_layers = 1
```

```
lmbd_init = tf.ones(shape=(self.n_qubits * 4 * self.n_layers,))
self.lmbd = tf.Variable(
    initial_value=lmbd_init, dtype="float32", trainable=True, name="lambdas"
)
```

I have changed the shape of `lmbd_init` to (12,), and now the code work. This would probably help me complete my masters, so thanks a lot.